



Real-Time Collaboration Platform

****Production-Ready Multi-User Document Editor****

Built with modern full-stack best practices
Industry Project – LogicVeda Web Development Domain
March 2026

Title: Real-Time Collaboration Platform

subtitle: Enterprise-Grade Real-Time Multi-User Document Collaboration System

author: LogicVeda

prepared-for: LogicVeda – Web Development Domain

date: March 2026

Project Code: lv1-2026-03-01

version: 2.0

 **Executive Summary** A secure, scalable, real-time collaborative document platform that supports simultaneous editing, presence awareness, threaded comments, version history, granular permissions, and production-grade

deployment — demonstrating end-to-end ownership of a complex full-stack application.

1. Project Overview & Business Value

Objective

Deliver a high-fidelity alternative to tools like Google Docs, Etherpad, Notion (lite), or Coda — suitable for remote teams, educational use, freelance collaboration, and startup MVPs.

Target Users

- Remote software teams
- University project groups
- Content creators / writers
- Consultants & agencies

Core Business Value Delivered

- Reduces email ping-pong & version confusion
- Enables instant feedback loops
- Demonstrates deep understanding of real-time systems, authentication, authorization, performance, security & DevOps

Non-Functional Requirements (NFRs)

- Latency: < 200 ms end-to-end edit propagation (95th percentile)
- Concurrency: ≥ 50 simultaneous active editors per document
- Availability: 99.5% (excluding planned maintenance)
- Security: OWASP Top 10 2021 mitigation + secure defaults

2. Detailed Functional Requirements

ID	Feature	Description	Acceptance Criteria
F-01	User Authentication	Email/password + refresh tokens	Register, login, logout, token refresh, password reset flow
F-02	Document CRUD	Create, list, read, update, delete documents	Only owner can delete; soft-delete support

ID	Feature	Description	Acceptance Criteria
F-03	Real-time Editing	Simultaneous editing with conflict resolution	Operational Transformation or Yjs/CRDT-like behavior; no lost characters
F-04	User Presence	Show live cursors with names & colors	Cursor appears/disappears on join/leave; color persists per session
F-05	Inline Threaded Comments	Add, reply, resolve, mention users in comments	Comments linked to text range; @mentions trigger notifications
F-06	Version History	Automatic snapshots + manual save points	View timeline, diff view (char/word level), restore any version
F-07	Role-Based Access Control (RBAC)	Owner, Editor, Viewer, Commenter	Granular permissions per document; invite via email/link
F-08	Notifications	In-app + email for comments, mentions, access grants	Real-time toast + async email delivery
F-09	Search & Filtering	Search documents by title/content	Full-text search (MongoDB text index or simple LIKE)
F-10	Offline Awareness	Visual indication when connection lost	Show warning banner; attempt reconnect every 5 s

3. Technology Stack – 2026 Production Grade

Category	Selection	Rationale / Alternatives Considered
Frontend Framework	React 19 (if avail.) / 18.3 • TypeScript 5.5+	Vite • Next.js considered but Vite chosen for speed
UI Component Library	MUI Joy / Material UI v6	shadcn/ui, Radix + Tailwind also evaluated
Rich Text Editor	Tiptap (ProseMirror) or Quill.js	Tiptap preferred for better collaboration extensions
Real-time Transport	Socket.io v4.8+	WebSocket + HTTP fallback; Yjs + y-websocket considered
Backend	Node.js 22 • Express 4.19+ • TypeScript	Fastify / NestJS evaluated – Express chosen for simplicity
Authentication	JWT (access + refresh) + Argon2 / bcrypt	OAuth2 / Lucia / Clerk considered
Primary Database	MongoDB 8.0 • Mongoose 8+	PostgreSQL + JSONB also viable

Category	Selection	Rationale / Alternatives Considered
Cache & Pub/Sub	Redis 7.4+ (ioredis client)	Dragonfly / KeyDB compatible
Email Delivery	Resend / SendGrid / Nodemailer + SMTP	Transactional + rate-limited
Containerization	Docker 27+ multi-stage	Podman compatibility
Orchestration	Kubernetes (basic manifests)	Docker Compose + Nomad also possible
CI/CD	GitHub Actions (matrix testing)	GitLab CI / CircleCI alternatives
Monitoring	Prometheus endpoint + Grafana (optional)	OpenTelemetry + Jaeger considered
Testing	Vitest + React Testing Library + MSW	Playwright E2E + Artillery load

4. Extremely Granular 28-Day Execution Plan

Week 1 – Planning, Setup, Authentication & Document CRUD

Day 1

- Create detailed requirements Notion page
- User stories + acceptance criteria
- Competitor feature matrix (Docs / Notion / Coda / Slab)

Day 2

- High-level C4 architecture diagram (Context + Containers)
- Data model ERD (User, Document, Permission, Comment, Version)
- Wireframes: login, dashboard, editor layout

Day 3

- Monorepo setup (pnpm workspaces)
- Backend boilerplate: Express + TS + ESLint + Prettier + Husky
- Folder structure decision + .env.example

Day 4

- Authentication domain:
 - Register → hash password → create user
 - Login → issue access + refresh token

- Refresh endpoint
- Logout (blacklist refresh token – Redis)

Day 5

- MongoDB Atlas / local setup
- Mongoose models + indexes
- Document service layer + CRUD controllers
- Ownership middleware

Day 6

- Socket.io integration
- Authentication via socket handshake (JWT in cookie / auth header)
- Room joining logic per documentId

Day 7

- Swagger OpenAPI v3 spec
- Postman collection export
- Week 1 demo: auth + document create/list/read

Week 2 – Real-time Editing Engine & Presence

Day 8–9

- Vite + React + TS project
- Protected routes • Auth context • Zustand store

Day 10

- Dashboard: infinite scroll document list
- Create document modal + title + initial content

Day 11–12

- Tiptap editor setup (or Quill)
- Collaborative extension (if Tiptap: @tiptap/extension-collaboration)
- Delta → JSON → Socket.io emit

Day 13

- Multi-cursor implementation
- User color generation (consistent per userId)
- Name badge above cursor

Day 14

- Smoke test: 3–4 browser tabs editing simultaneously
- Fix desync / lost characters issues

Week 3 – Comments, Versions, RBAC, Notifications

Day 15

- Permission model: document → array of {userId, role}
- Invite flow (email link with token)

Day 16

- Versioning strategy: diff-based or full snapshot every 30s
- History UI: timeline slider + side-by-side diff

Day 17

- Comment domain: range-based anchors
- Threaded replies • resolve / reopen

Day 18

- Redis: online users set per document
- Pub/sub channel per document for fast broadcasts

Day 19

- Notification service:
 - In-app (Socket.io)
 - Email queue (BullMQ + Redis)

Day 20

- Security week:
 - Helmet, CORS strict, rate-limit
 - Input sanitization (DOMPurify + allow-list)
 - JWT validation edge cases

Day 21

- Load test: 50 concurrent editors (Artillery)
- Fix bottlenecks (MongoDB indexes, Redis pipelining)

Week 4 – Deployment, Observability & Polish

Day 22

- Multi-stage Dockerfiles (frontend + backend)
- docker-compose.prod.yml

Day 23

- Kubernetes manifests: Deployment, Service, Ingress, HPA
- minikube / kind local validation

Day 24

- GitHub Actions matrix:
 - lint • test • build • docker build & push

Day 25

- Deploy: Vercel (frontend) + Render/Railway/Fly (backend + Redis + Mongo)
- Custom domain + Let's Encrypt HTTPS

Day 26

- Basic health endpoints (/health, /metrics)
- UptimeRobot / Pingdom monitoring

Day 27

- Record demo: 5–7 min walkthrough (scenario-based)
- Voiceover + screen annotations

Day 28

- Final QA checklist (cross-browser, mobile responsive)
- README.md: architecture, setup, screenshots, video link
- Export this document to PDF

LogicVeda – Web Development Domain

Project Submission Guidelines

March 2026 Edition

Prepared for participants

Focus: Industry-grade full-stack / advanced web projects

Submission period: You need to submit the project on or before the due date if you submit after the due date means you will be automatically disqualified for stipend by the system.

Evaluation emphasis: Code quality · Documentation · Live demo · Scalability & security awareness

1. General Rules & Eligibility

- All code must be **original work** created mainly during the LogicVeda preparation/submission window.
- Participants must submit **exactly 1 project** (as per the planned structure: 1 month each, weekly breakdown).
- Projects should demonstrate **progressive complexity**:
 1. Strong modern frontend / real-time foundations
 2. Distributed systems / microservices / resilience
 3. AI/ML-integrated or data-heavy web application
- Plagiarism / Ai generated content will result in disqualification for stipend.

2. Mandatory Submission Deliverables

Submit **one consolidated package** containing all three projects:

#	Deliverable	Format / Location	Required?	Evaluation Weight
1	Project Documentation / Report (1 PDF)	1 PDF file (A4, 8–15 pages)	Yes	25%
2	Live Public Demo URL	1 HTTPS link (no mandatory login)	Yes	30%
3	GitHub Repository	1 repo with clear folders	Yes	20%
4	README.md per project	Detailed, professional README in repo	Yes	15%
5	Demo Video(s)	3–7 min per project (YouTube unlisted / Loom / Drive)	Yes	10%

Naming convention recommendation

`YourName_Project1_RealTimeCollab_LogicVeda_March2026.pdf/zip`

3. Documentation Guidelines – What every PDF should contain

Use consistent structure across all three documents for a polished portfolio feel.

1. Hero / Cover Section

- Large project title
- Catchy tagline
- Your name + @harsadash + date

- Gradient background (optional but recommended)

2. Project Overview

- Vision & objectives
- Target users / use cases
- Business value delivered
- Non-functional goals (latency, concurrency, availability targets)

3. Key Features (table format)

| ID | Feature | Description | Acceptance Criteria |

4. Technology Stack (table)

| Category | Technology | Rationale / Alternatives |

5. Architecture Diagram

- Include screenshot (Excalidraw, Draw.io, Lucidchart, etc.)
- Or clean ASCII art if no image tool used

6. Detailed Execution Timeline

- Day-by-day or week-by-week breakdown
- Major deliverables per phase
- Milestones & checkpoints

7. Technical Highlights

- Security measures (OWASP mitigation, input sanitization, rate limiting, etc.)
- Performance & scalability notes (caching, lazy loading, load test results)
- Challenges faced & how solved

8. Deployment & Operations

- Platform used (Vercel, Render, Railway, AWS, Fly.io, etc.)
- CI/CD pipeline summary
- Monitoring / health checks (if implemented)

9. Visuals

- 5-10 high-quality screenshots / GIFs
- Architecture diagram
- Live demo captures

10. Personal Reflection (optional but strongly recommended)

- Key learnings
- Industry best practices applied
- Future roadmap ideas

4. Code & Repository Expectations

- Clean, modular code structure
- Consistent naming & formatting (use ESLint / Prettier)
- Semantic commit messages (`feat: add real-time cursor presence` , `fix: JWT refresh token bug` , etc.)
- Feature branches & pull requests (even if solo)
- `.gitignore` properly configured
- No committed secrets / API keys
- Basic tests appreciated (even 30–50% coverage shows intent)
- Responsive design + accessibility basics (ARIA labels, keyboard navigation)

5. Live Demo Requirements

- Publicly accessible (no VPN / geo-restriction)
- HTTPS enforced
- Core functionality available without sign-up (demo account OK if needed)
- Fast initial load (< 5 seconds preferred)
- Mobile responsive (test on phone / tablet)
- Include brief instructions on demo page if non-obvious

6. Evaluation Criteria (Suggested 100-point scale)

Category	Points	Focus Areas
Innovation & Problem Solving	15	Original features, thoughtful design choices
Technical Depth & Best Practices	25	Clean code, modern stack, security & performance awareness
Functionality & User Experience	20	Works as promised, intuitive UX, responsive
Documentation Quality	20	Clear, professional, well-structured
Deployment & Reliability	10	Stable live demo, easy local setup

Category	Points	Focus Areas
Presentation & Polish	10	Demo video quality, visual appeal of docs

7. Important Rules & Tips

- **Strict deadline** – no late submissions accepted
- **File size limits:** ZIP ≤ 500 MB, individual PDFs ≤ 10 MB
- **No external dependencies that require payment / private keys** for judges to run demo
- **Backup plan:** If live demo is down, high-quality video + screenshots must still convey full functionality
- **Stand-out practices:**
 - Include Lighthouse scores / performance metrics
 - Show real multi-user session in video (especially Project 1)
 - Mention OWASP mitigations, load test results, CI/CD pipeline
 - End docs with personal growth reflection

Submission email / form subject example

Thers no email submission you need to submit the project in dashboard itself

Good luck!

Submit with confidence – your detailed planning and advanced topics already make these projects stand out.

Team LogicVeda

March 2026

Crafted with precision and modern engineering principles • LogicVeda Technologies • March 2026

